

1 Formalization in TypeSpec

Annotations	A	::=	$\overline{@spec\ E}$
Expressions	E	::=	$x(\bar{T}) :: T \mid x(\bar{T}) :: T \text{ when } \overline{\alpha : \bar{T}}$
Types	T	::=	$\alpha \mid T \mid T$ $\mid \text{any}() \mid \text{none}()$ $\mid \text{atom}() \mid :k$ $\mid \text{pid}() \mid \text{port}() \mid \text{reference}()$ $\mid \text{float}() \mid \text{integer}()$ $\mid \text{neg_integer}() \mid \text{non_neg_integer}() \mid \text{pos_integer}()$ $\mid \text{tuple}() \mid \{\bar{T}\} \mid \% \{\bar{F}\}$ $\mid [] \mid \text{nonempty_maybe_improper_list}(T, T)$ $\mid \llcorner \mid \llcorner :: n \llcorner \mid \llcorner :: *n \llcorner \mid \llcorner :: n, _ :: *n \llcorner$ $\mid (\bar{T} \rightarrow T) \mid (\dots \rightarrow T)$ $\mid n \mid n..n'$
Fields	F	::=	$\text{required}(T) \Rightarrow T \mid \text{optional}(T) \Rightarrow T$

where x ranges over expression variables, α ranges over type variables, n ranges over integers with $n < n'$, and k ranges over atom names (N.B., the notation $\overline{E}$ denotes a possibly empty sequence of elements E separated by commas apart from $\overline{@spec\ E}$ and $\overline{\alpha : \bar{T}}$, where it denotes a non-empty sequence.)

Figure 1: TypeSpec

1.1 TypeSpec Built-in Types and Syntactic Sugars

Built-in types	Defined as
term()	any()
arity()	0..255
*as_boolean(T)	T
binary()	«_::_*8»
nonempty_binary()	«_::_8, _::_*8»
bitstring()	«_::_*1»
nonempty_bitstring()	«_::_1, _::_*1»
boolean()	:true :false
byte()	0..255
nonempty_list(T)	nonempty_maybe_improper_list(T, [])
list(T)	[] nonempty_list(T)
nonempty_list()	nonempty_list(any())
*nonempty_improper_list(T ₁ , T ₂)	nonempty_maybe_improper_list(T ₁ , T ₂)
maybe_improper_list(T ₁ , T ₂)	[] nonempty_improper_list(T ₁ , T ₂)
maybe_improper_list()	maybe_improper_list(any(), any())
nonempty_maybe_improper_list()	nonempty_maybe_improper_list(any(), any())
char()	0..0x10FFFF
charlist()	list(char())
nonempty_charlist()	nonempty_list(char())
fun()	(... -> any())
function()	fun()
identifier()	pid() port() reference()
iodata()	iolist() binary()
iolist()	maybe_improper_list(byte() binary() iolist(), binary() [])
keyword()	[{atom(), any()}]
keyword(T)	[{atom(), T}]
mfa()	{module(), atom(), arity()}
module()	atom()
no_return()	none()
node()	atom()
number()	integer() float()
struct()	%{:_struct__ => atom(), optional(atom()) => any()}
timeout()	:infinity non_neg_integer()

(built-in types with prepending asterisk (*) are only listed for documentation)

Table 1: TypeSpec Built-in Types

Syntactic sugar	Expanded as
true	:true
false	:false
nil	:nil
[T, ...]	nonempty_list(T)
[T]	list(T)
T ₁ => T ₂	required(T ₁) => T ₂
k:T	:k => T
%k{F}	%{:__struct__ => :k, F}

Table 2: TypeSpec Syntactic Sugars

2 Formalization in Elixir Type

Annotations	A ::= \$S \$S when $\overline{\alpha:T}$	
Signatures	S ::= $\overline{T} \rightarrow T$ S and S	
Types	T ::= α K term() none() dynamic() T or T T and T not T n..n infty..n n..infty { $\overline{T}$ } { $\overline{T}, \dots$ } %{ $\overline{F}$ } %{ $\dots, \overline{F}$ } ($\overline{T} \rightarrow T$) empty_list() non_empty_list(T, T)	type variables basic types type connectives integers (internal representation) type constructors
Key types	K ::= atom() :k pid() port() reference() float() integer() binary() tuple() open_map() fun() list()	
Fields	F ::= :k=>T K=>if_set(T)	

where x ranges over expression variables, α ranges over type variables, n ranges over integers (integers internal representation: types not available to the programmer), and k ranges over atom names. (N.B., the notation $\overline{E}$ denotes a possibly empty sequence of elements E separated by commas apart from { $\overline{T}, \dots$ }, %{ $\dots, \overline{F}$ }, and $\overline{\alpha:T}$, where it denotes a non-empty sequence. Also, in $\overline{F}$ each F must have a different keytype)

Figure 2: Elixir Type

2.1 Elixir Type Built-in Types and Syntactic Sugars

Built-in types	Defined as
boolean()	:true or :false
non_empty_list(T)	non_empty_list(T, empty_list())
list(T, T')	empty_list() or non_empty_list(T, T')
list(T)	empty_list() or non_empty_list(T)
list()	empty_list() or non_empty_list(term())

Table 3: Elixir Type Built-in Types

Syntactic sugar	Expanded as
$\{\dots\}$	<code>tuple()</code>
$\%\{\dots\}$	<code>open_map()</code>
$k : T$	<code>:k => T</code>
<code>K => not_set()</code>	<code>K => if_set(none())</code>
<code>K => T</code> <i>(for $K \neq :k$)</i>	<code>K => if_set(T)</code>

Table 4: Elixir Type Syntactic Sugars

3 Translation from TypeSpec to Elixir Type

$\llbracket \cdot \rrbracket : \text{TypeSpec} \rightarrow \text{Elixir type}$

Figure 3: Semantic of Translation

3.1 Translation in Annotations

$$\llbracket @spec\ E \rrbracket = \$ \llbracket E \rrbracket$$

$$\llbracket @spec\ \bar{E}, @spec\ E' \rrbracket = \$ \llbracket \bar{E}, E' \rrbracket$$

$$\llbracket x(\bar{T}_1) :: T_2 \rrbracket = \llbracket \bar{T}_1 \rrbracket \rightarrow \llbracket T_2 \rrbracket$$

$$\llbracket x(\bar{T}_1) :: T_2 \text{ when } \bar{\alpha} : \bar{T}_3 \rrbracket = \llbracket \bar{T}_1 \rrbracket \rightarrow \llbracket T_2 \rrbracket \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket$$

$$\llbracket \bar{E}, x(\bar{T}_1) :: T'_2 \rrbracket = \begin{cases} S \text{ and } \llbracket \bar{T}'_1 \rrbracket \rightarrow \llbracket T'_2 \rrbracket \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket & \text{if } \llbracket \bar{E} \rrbracket \approx S \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket \\ \llbracket \bar{E} \rrbracket \text{ and } \llbracket \bar{T}'_1 \rrbracket \rightarrow \llbracket T'_2 \rrbracket & \text{otherwise} \end{cases}$$

$$\llbracket \bar{E}, x(\bar{T}'_1) :: T'_2 \text{ when } \bar{\alpha}' : \bar{T}'_3 \rrbracket = \begin{cases} S \text{ and } \llbracket \bar{T}'_1 \rrbracket \rightarrow \llbracket T'_2 \rrbracket \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket, \bar{\alpha}' : \llbracket \bar{T}'_3 \rrbracket & \text{if } \llbracket \bar{E} \rrbracket \approx S \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket \\ & \text{and } \bar{\alpha} \cap \bar{\alpha}' = \emptyset \\ S \text{ and } \llbracket \bar{T}'_1 \rrbracket \rightarrow \llbracket T'_2 \rrbracket[\alpha' := \beta'] \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket, \bar{\beta}' : \llbracket \bar{T}'_3 \rrbracket & \text{if } \llbracket \bar{E} \rrbracket \approx S \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket \\ & \text{and } \bar{\alpha} \cap \bar{\alpha}' \neq \emptyset \\ \llbracket \bar{E} \rrbracket \text{ and } \llbracket \bar{T}'_1 \rrbracket \rightarrow \llbracket T'_2 \rrbracket \text{ when } \bar{\alpha}' : \llbracket \bar{T}'_3 \rrbracket & \text{otherwise} \end{cases}$$

Figure 4: Translation in Annotations

3.2 Translation in Types

```

[[α]] = α
[[T1 | T2]] = [[T1]] or [[T2]]
[[any()]] = term()
[[none()]] = none()
[[atom()]] = atom()
[[ :k ]] = :k
[[pid()]] = pid()
[[port()]] = port()
[[reference()]] = reference()
[[float()]] = float()
[[integer()]] = integer()
[[neg_integer]] = infy..-1
[[non_neg_integer]] = 0..infy
[[pos_integer]] = 1..infy
[[tuple()]] = tuple()
[[{T̄}]] = {[T̄]}
[[%{F1, ..., Fn}]] = map(⊕i=1n[[Fi]])
[[[]]] = empty_list()
[[nonempty_maybe_improper_list(T1, T2)]] = non_empty_list([[T1]], [[T2]])
[[«»]] = binary()
[[«_:n»]] = binary()
[[«_:_*n»]] = binary()
[[«_:n, _:_*n»]] = binary()
[[({T̄} -> T)]] = ([[T̄]] -> [[T]])
[[(... -> T)]] = { fun() if [[T]] ≈ term()
                  fun() and dynamic() otherwise
[[n]] = n..n
[[n..n']] = n..n'

```

Figure 5: Translation in Types

3.3 Translation in Fields

Let τ range over the right-hand side type of field types, that is, τ is either T or $\text{if_set}(T)$. We consider lists whose elements are of the form $K \Rightarrow \tau$. Given such a list L we write $K \in L$ if there exist τ such that the element $K \Rightarrow \tau$ belongs to L . We denote by **Empty** the lists that maps all the non-singleton keytypes to **not_set()**.

Given two field lists L_1 and L_2 , we write $L_1 \oplus L_2$ the list obtained by adding to L_1 all the fields elements $K \Rightarrow \tau$ of L_2 such that $K \notin L_1$. Formally:

$$\begin{aligned}
L_1 \oplus [] &= L_1 \\
L_1 \oplus [K \Rightarrow \tau, L_2] &= L_1 \oplus L_2 && \text{if } K \in L_1 \\
L_1 \oplus [K \Rightarrow \tau, L_2] &= [K \Rightarrow \tau, L_1] \oplus L_2 && \text{if } K \notin L_1
\end{aligned}$$

Figure 6: Merge in Field Types

We then define $\text{map}(L)$ as the map type whose field list is L . Then we have $\llbracket \% \{ F_1, \dots, F_n \} \rrbracket = \text{map}(\bigoplus_{i=1}^n \llbracket F_i \rrbracket)$. If we want a saturated map type (i.e., a map type that specifies all the possible keytypes), then we have $\llbracket \% \{ F_1, \dots, F_n \} \rrbracket = \text{map}(\bigoplus_{i=1}^n \llbracket F_i \rrbracket \oplus \text{Empty})$.

All it remains to do is to define the translation of field types. If $K_1, \dots, K_n$ are pairwise distinct key types, we write $\llbracket \bigvee_{i=1}^n K_i \Rightarrow \tau \rrbracket$ for the list $[K_1 \Rightarrow \tau, \dots, K_n \Rightarrow \tau]$. Then the translation of single field types is defined in Figure 7.:

$$\begin{aligned}
\llbracket \text{required}(T) \Rightarrow T' \rrbracket &= \begin{cases} [:k \Rightarrow \llbracket T' \rrbracket] & \text{if } \llbracket T \rrbracket \approx :k \\ [\bigvee_{i \in I} K_i \Rightarrow \text{if_set}(\llbracket T' \rrbracket)] & \text{if } \llbracket T \rrbracket \approx \bigvee_{i \in I} K_i \text{ and } \llbracket T \rrbracket \not\approx :k \\ \text{undefined} & \text{otherwise} \end{cases} \\
\llbracket \text{optional}(T) \Rightarrow T' \rrbracket &= \begin{cases} [\bigvee_{i \in I} K_i \Rightarrow \text{if_set}(\llbracket T' \rrbracket)] & \text{if } \llbracket T \rrbracket \approx \bigvee_{i \in I} K_i \\ \text{undefined} & \text{otherwise} \end{cases}
\end{aligned}$$

where each K_i in the list is pairwise distinct to one another.

Figure 7: Translation in Field Types